PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Contents

1. Project Overview 3
2. Background & Business Objective 4
3. Product Purpose 5
4. System Functionalities 6
5. User Stories & Use Cases 12
6. Features Included (Scope Definition) 14
7. Features Not Included (Scope Control) 14
8. Assumptions & Constraints 14
9. Risks & Questions Throughout Development 15

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: TreasurAI - AI Financial Decision System

Version: 1.0

1. Project Overview

- **Problem Statement:**
  Modern financial teams rely heavily on static dashboards and manual analysis when making treasury decisions such as currency conversion, payment scheduling, and supplier negotiation. These approaches are time-consuming, reactive, and lack actionable insights. Furthermore, existing systems do not provide clear recommendations or explain the reasoning behind financial decisions, leading to inefficiencies and potential financial losses.

- **Target Domain:**
  The system is designed for the **financial and treasury management domains,** focusing on:
  - Foreign exchange (FX) decision-making
  - Cash flow management
  - Supplier pricing optimization
  - Financial risk analysis

- **Proposed Solution Summary:**
  TreasurAI is an AI-powered financial decision platform that transforms structured financial data and real-time market intelligence into **actionable recommendations.**
  The system enables users to:
  - Receive AI-generated financial decisions
  - Understand the reasoning behind each decision

© UMHackathon 2026
Email: umhackathon@um.edu.my

- Simulate outcomes before execution
- Quantify financial impact in terms of potential gain/loss direction and risk

2. Background & Business Objective

- **Background of the Problem:**

  Traditional financial systems primarily serve as data visualization tools. While they present financial metrics such as cash flow and exchange rates, they do not actively guide users in making optimal decisions.

  Decision-making is often:

  - Manual and time-consuming
  - Dependent on human expertise
  - Lacking real-time external context (e.g., market news, economic signals)

- **Importance of Solving This Issue:**

  By addressing these limitations, TreasurAI aims to:

  - Reduce financial losses caused by poor time decisions
  - Improve operational efficiency
  - Enable faster and more informed decision-making
  - Increase transparency through explainable AI

- **Strategic Fit / Impact:**

  TreasurAI aligns with the growing trend of **AI-driven enterprise systems,** where artificial intelligence is used not only for prediction but also for decision support and automation.

  It demonstrates how large language models (GLMs) can be integrated into financial systems to deliver real business value through:

  - Intelligent recommendations
  - Context-aware reasoning
  - Actionable and explainable outcomes

© UMHackathon 2026

3. Product Purpose

3.1. Main Goal of the System:

To provide an **AI-powered decision assistance system** that helps organizations make better financial decisions by combining internal financial data with external market intelligence.

3.2. Intended Users (Target Audience):

- Finance managers
- Treasury teams
- Chief financial officers (CFOs)
- Business analysts
- Enterprise decision makers

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. System Functionalities

4.1. Description:

TreasurAI operates as an **intelligent decision engine** that integrates multiple data sources and uses AI to generate actionable insights.

The system workflow includes:

a. Collecting structured financial data (ERP-like data)
b. Integrating unstructured data (news, FX signals)
c. Processing data through a GLM-based decision engine
d. Delivering recommendations via an interactive dashboard

4.2. Key Functionalities:

a. AI Decision Feed

This system presents a centralized feed of AI-generated financial recommendations. Each recommendation includes an action title, a summary of reasoning, impact (gain/loss/neutral), risk level, and confidence score.

Users can interact with each recommendation through:

- "Why this decision" (view explanation)
- "Show changes" (simulate impact)
- "Take action" (execute decision)
- "Dismiss" (remove recommendation)

b. Explainable AI (XAI)

Each recommendation includes a detailed explanation that consists of:

- Internal financial data used
- External market signals
- Logical reasoning behind the decision

This ensures transparency and builds trust in AI-generated outputs.

© UMHackathon 2026

c. Cash Flow Visualization & Simulation

The system provides an interactive cash flow timeline displaying net cash position only (no separate payables/receivables lines). AI decision markers (convert, delay, hedge) are not overlaid on the chart).

When users select "Show changes" (either single decision or "Show All Decisions"):

- The chart updates automatically
- A solid cyan line represents the optimized net cash position
- The dashed yellow line represents the original net cash position
- Both lines can be compared side-by-side

The feature is accessed through the "Show Changes" button on each recommendation card, or the "Show Comparison" toggle button above the chart.

d. Floating Live Intelligence Window

A floating, collapsible window provides real-time contextual information, including FX exchange rates and market news updates. This window can be opened or closed by the user without affecting the main interface.

e. Trade-off & ROI Simulator

The system allows users to compare different financial strategies, such as convert now vs delay, hedge vs no hedge, and negotiate vs accept. Its outputs will include impact direction, risk level, and strategy comparison.

f. Smart Negotiation Generator

When supplier overpricing is detected, the system generates price comparison (current vs market average), suggestion discount range, and negotiation strategy.

g. Fallback Intelligence Mode

If the AI model fails or becomes unavailable, the system switches to fallback mode. Recommendations are generated using historical data and predefined logic.

© UMHackathon 2026

Email: umhackathon@um.edu.my

h. AI Configuration Panel

Users can configure system behavior through:

- Risk calibration slider (conservation to aggressive)
- API connection status monitoring
- Threshold-based alerts
- User permission controls

4.3. AI Model & Prompt Design

4.3.1. Model Selection:

The system uses **ILMU AI** as the core reasoning engine due to its strong capabilities in:

Criteria	Description
Advanced reasoning and decision-making	Able to analyze complex financial scenarios (FX, payments, supplier pricing) and generate actionable recommendations instead of simple summaries
Integration of structured and unstructured data	Combines internal financial data (cash flow, payables, receivables) with external data (financial news, FX trends) for context-aware decisions
Structured JSON output generation	Generates consistent JSON responses, enabling smooth integration with backend APIs and frontend UI rendering
Explainable reasoning support	Produces reasoning chains that allow the system to display transparent explanations through the “Why this decision” feature
Multi-scenario analysis capability	Supports comparison of different strategies (e.g., convert now vs delay vs hedge) for trade-off and impact analysis

© UMHackathon 2026

4.3.2. Prompting Strategy:

This system uses a **structured zero-shot prompting strategy** to guide the ILMU AI in generating financial decisions. This approach is chosen as it allows the model to perform **flexible reasoning across dynamic financial scenarios** without relying on predefined examples.

The model is instructed to act as a **financial treasury strategist,** producing actionable recommendations such as currency conversion, payment delay, or supplier negotiation. Both **structured data** (cash flow, payables, receivables) and **unstructured data** (financial news, FX signals) are included to provide complete context.

Clear instructions are applied to ensure the model generates **relevant decisions with reasoning,** while avoiding vague outputs. Responses are constrained to a **structured JSON format,** enabling consistent integration with backend systems and frontend display.

This strategy is well-suited for TreasurAI as it supports **real-time decision-making, explainability, and multi-scenario analysis,** which are essential for financial optimization.

4.3.3. Context & Input Handling:

Aspect	Description
Input types	The system processes both **structured data** (cash balance, payables, receivables, currency exposure) and **unstructured data** (financial news, FX signals, economic updates).
Data Integration	All inputs are merged into a **unified prompt structure** before being sent to the ILMU AI, ensuring full financial and market

© UMHackathon 2026

	context is preserved.
Data Cleaning	Irrelevant, duplicate, or non-financial information is filtered out to improve relevance and reduce noise in model input.
Data Normalization	Inputs are standardized into consistent formats (e.g., currency format, date format, numerical alignment) to ensure accurate interpretation.
Input Size Control	The system currently uses a fixed input size with a small number of news items (max 3) to keep prompts manageable. No automatic token counting, truncation, or summarization is performed before sending to the GLM.
Oversized Input Handling	
Context Prioritization	Critical data such as urgent payments, large transactions, and high-volatility FX news are prioritized in prompt construction.
Real-time Refresh	Data is refreshed on-demand when the dashboard loads or when the user changes scenarios. FX rates fetched from live APIs. news is fetched from RSS feeds when available.

4.3.4. Fallback & Failure Behavior:

Scenario	**System Behavior**	**Recovery Strategy**
GLM API Timeout	Request is terminated and fallback mode is activated	Retry request + use cached/historical logic
GLM API Failure	System bypasses GLM and activates fallback engine	Retry API + log error
Invalid JSON Output	Response is rejected by validation layer	Immediately falls back to historical/mock data
Hallucinated Output	Output is validated against schema and context	Filter response + fallback logic

© UMHackathon 2026

Missing Data	System detects incomplete inputs	Use historical averages
Oversized Input	Input is truncated or summarized before processing	Auto preprocessing pipeline
Data Sync Failure	ERP/news data fails to refresh	Background retry sync
Full System Failure	Multiple services fail simultaneously	Rule-based decision engine activated

© UMHackathon 2026

Email: umhackathon@um.edu.my

5. User Stories & Use Cases

- **User Stories:**
  - As a CFO, I want to receive AI-driven financial recommendations so that I can make better currency conversion and treasury decisions.
  - As a treasury analyst, I want to compare different financial strategies (convert, delay, hedge) so that I can evaluate risk and potential financial impact before making decisions.
  - As a finance manager, I want to understand why the AI made a decision so that I can trust and validate its recommendation.
  - As a user, I want to view real-time FX rates and market news so that I can make informed financial decisions.
  - As a decision maker, I want to simulate different financial scenarios so that I can predict outcomes before executing actions.

- **Use Cases (Main Interactions):**

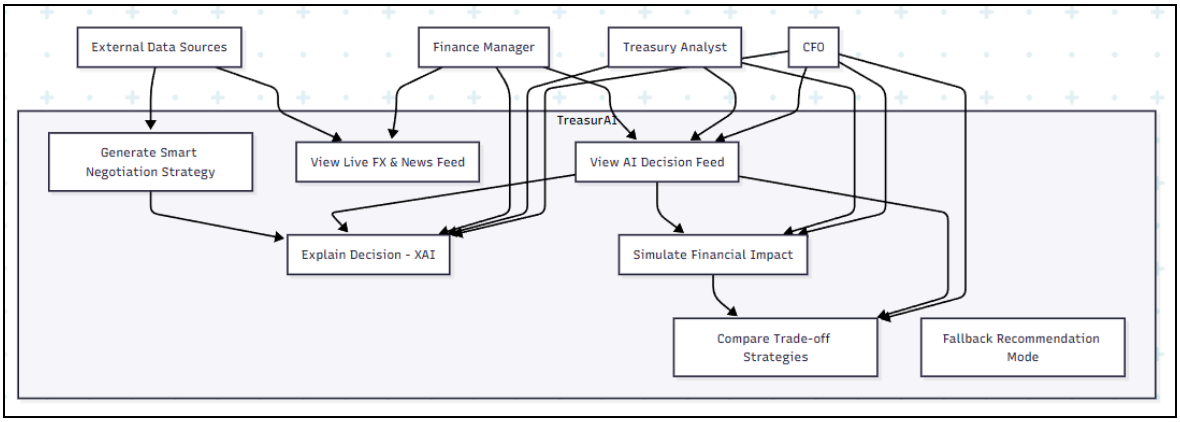
Use Cases	**Steps**
AI Decision Feed Generation	• User opens dashboard • System fetches financial + market data • ILMU AI generates recommendations (convert/delay/negotiate) • Results are displayed in Decision Feed
Explainable AI	• User clicks “Why this decision” • System retrieves GLM reasoning output • System displays market news used, internal financial data used, and reasoning chain
Trade-off & ROI Simulation	• User selects “Show Changes” • System scrolls to cash flow comparison chart • System displays before vs after financial impact and risk

© UMHackathon 2026

Email: umhackathon@um.edu.my

	difference
Smart Negotiation Generation	• System detects supplier overpricing • GLM generates negotiation suggestion • User views recommended discount strategy • System displays actionable negotiation script
Real-Time Intelligence Feed	• Floating FX window shows live updates • User can close or reopen panel anytime • Includes FX rates and market news • Updates dynamically without refreshing page

- **Use Case Diagram:**



© UMHackathon 2026
Email: umhackathon@um.edu.my

6. Features Included (Scope Definition)

- AI Decision Feed
- Explainable AI panel
- Cash flow visualization with simulation
- Floating intelligence window
- Trade-off and ROI simulator
- Negotiation generator
- Fallback intelligence mode

7. Features Not Included (Scope Control)

- Real ERP system integration
- Automated execution of financial transactions
- Multi-user collaboration
- Production-level deployment
- Advanced financial modeling

8. Assumptions & Constraints

- **GLM Cost Constraint:**
  - Prompt size is optimized to reduce token usage
  - Repeated queries may be cached
  - Input data is filtered before processing
- **Technical Constraints:**
  - System uses mock data for demonstration
  - Limited integration with real external APIs
- **Performance Constraints**
  - AI response time may vary depending on load
  - Simulation uses simplified models

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **User Input:**
  - Requires structured financial input
  - Dependent on data quality

9. Risks & Questions Throughout Development

- **Design Similarity:**

  How are we ensuring that GLM does not generate similar financial decisions across different users or scenarios?

  → Dynamic inputs (FX rates, cash flow data, market news) are used to ensure each recommendation is context-specific and unique.
- **Decision Accuracy & Reliability**

  How do we ensure AI financial decisions are safe and correct?

  → All GLM outputs are validated using structured JSON format and checked against financial constraints (cash balance, risk level, exposure limits).
- **Hallucination Handling**

  What happens if GLM generates incorrect or unsupported financial reasoning?

  → A validation layer filters invalid outputs, and fallback mode replaces them with safe historical-based recommendations.
- **Data & External Dependency Failure**

  What if external APIs (FX news, ERP data) are unavailable?

  → The system switches to fallback intelligence mode using cached and historical data to maintain functionality.
- **User Trust & Explainability**

  How do we ensure users trust AI-generated financial decisions?

  → Every output includes an Explainable AI (XAI) section showing reasoning, data sources, and expected financial impact.

© UMHackathon 2026